سوال سوم) پردازش متن و صوت:

صوت:

بخش اول:

در این مرحله، هدف آماده‌سازی مجموعه داده‌های صوتی مختلف برای استفاده در مدل‌های تشخیص احساسات صوتی است. برای این منظور، از چهار دیتاست رایج در حوزه‌ی تشخیص احساسات صوتی استفاده شده است: CREMA-D، RAVDESS، SAVEE و TESS.

مراحل انجام‌شده:

- ابتدا مسیر اصلی داده‌ها تعریف شده و با استفاده از توابع جداگانه، هر دیتاست به صورت مجزا پردازش شده است.
- در هر تابع، نام فایل‌ها بر اساس ساختار نام‌گذاری خاص هر دیتاست تجزیه شده و احساسات موجود استخراج شده‌اند.
- برای جلوگیری از ناهماهنگی در برچسب‌های احساسی، یک دیکشنری استاندارد برای نگاشت احساسات به کدهای ثابت (مثلاً ANG برای خشم یا NEU برای حالت خنثی) استفاده شده است.
- به منظور تطبیق برچسب‌ها حتی در صورت اختلاف جزئی در نوشتار (مثلاً "angry" به جای "anger") از الگوریتم تطبیق مبهم (fuzzy matching) کمک گرفته شده است.
- داده‌های خروجی از هر دیتاست در قالب یک DataFrame پانداس ذخیره شده‌اند و در نهایت با یکدیگر ترکیب و به صورت تصادفی (shuffled) مرتب‌سازی شده‌اند تا از ترتیب احتمالی داده‌ها جلوگیری شود.

نکات:

- این مرحله به نحوی طراحی شده که داده‌ها با ساختاری یکسان و برچسب‌گذاری استاندارد وارد مراحل بعدی شوند.

- تنوع منابع (چهار دیتاست متفاوت) باعث افزایش توان تعمیم مدل و جلوگیری از overfitting به یک دیتاست خاص خواهد شد.
- توجه شده که تعداد نمونه‌ها در احساسات مختلف متعادل نیست که در ادامه باید مورد بررسی و احتمالاً بازنمونه‌گیری (resampling) قرار گیرد.

```
CREMA-D: 100%|██████████| 7442/7442 [00:00<00:00, 15020.39it/s]
RAVDESS: 100%|██████████| 1440/1440 [00:00<00:00, 10089.74it/s]
SAVEE: 100%|██████████| 480/480 [00:00<00:00, 159846.44it/s]
TESS: 100%|██████████| 14/14 [00:00<00:00, 309.07it/s]
Total samples: 12162
Emotions: emotion
ANG    1923
FEA    1923
DIS    1923
SAD    1923
HAP    1923
NEU    1895
SUR     652
Name: count, dtype: int64
Datasets included: dataset
CREMA      7442
TESS       2800
RAVDESS    1440
SAVEE       480
Name: count, dtype: int64
```

بخش دوم (تقسیم‌بندی داده‌ها به مجموعه‌های Train، Validation و Test):

پس از آماده‌سازی و تجمیع داده‌های صوتی، در این مرحله داده‌ها به سه مجموعه‌ی اصلی تقسیم شدند:

- **مجموعه آموزش(Train)** : برای آموزش مدل یادگیری ماشین.
- **مجموعه اعتبارسنجی(Validation)** : برای ارزیابی مدل در طول آموزش و تنظیم هایپراپارامترها.
- **مجموعه آزمون(Test)** : برای ارزیابی نهایی عملکرد مدل پس از آموزش کامل.

روش تقسیم‌بندی:

- ابتدا 70٪ از داده‌ها به صورت تصادفی به عنوان مجموعه آموزش انتخاب شد.
- مابقی 30٪ به دو قسمت مساوی برای اعتبارسنجی و آزمون تقسیم گردید (هر کدام 15٪).

- جهت اطمینان از یکنواختی، تقسیم‌بندی با مقدار random_state=42 انجام شده تا نتایج قابل بازتولید باشند.
- همچنین shuffle=True استفاده شده تا ترتیب داده‌ها تأثیری بر تقسیم نداشته باشد.

ذخیره‌سازی:

- هر مجموعه به صورت جداگانه در قالب فایل CSV ذخیره شده است تا در مراحل بعدی، مدل بتواند به سادگی به آن‌ها دسترسی پیدا کند.

تحلیل آماری:

- برای هر مجموعه، آمار مربوط به تعداد نمونه‌ها، توزیع برچسب‌های احساسی، و توزیع دیتاست‌های منبع چاپ شده است.
- این کار کمک می‌کند تا بررسی شود که توزیع احساسات بین مجموعه‌ها متعادل باشد و هیچ دیتاست خاصی در یکی از مجموعه‌ها غالب نباشد (که ممکن است باعث bias شود).

```
Train size: 8513
Train emotion distribution:
emotion
DIS    1360
SAD    1348
HAP    1346
NEU    1335
ANG    1333
FEA    1331
SUR     460
Name: count, dtype: int64
Train dataset distribution:
dataset
CREMA      5193
TESS       1978
RAVDESS     999
SAVEE       343
Name: count, dtype: int64

Validation size: 1824
Validation emotion distribution:
emotion
HAP    306
FEA    304
...
TESS        426
RAVDESS     229
SAVEE        71
Name: count, dtype: int64
```

بخش سوم (بارگذاری داده‌ها و آماده‌سازی مدلWav2Vec2.0):

در این مرحله، داده‌های صوتی آماده‌شده به کمک کتابخانه‌ی قدرتمند HuggingFace Datasetsبارگذاری و برای آموزش مدل مبتنی بر **Wav2Vec2.0** تنظیم شده‌اند.

مراحل انجام‌شده:

1. **بارگذاری داده‌ها:**
   - سه فایل CSV مربوط به مجموعه‌های آموزش، اعتبارسنجی و آزمون با مشخص‌کردن مسیر و تعیین جداکننده (tab) بارگذاری شدند.
   - نمونه‌هایی از داده‌ها بررسی شد تا از صحت ساختار فایل‌ها اطمینان حاصل شود.
2. **تعریف ستون‌ها:**
   - ستون حاوی مسیر فایل صوتی به عنوان ورودی مدل (path) و ستون احساس به عنوان خروجی مدل (emotion) تعیین گردید.
3. **استخراج برچسب‌های منحصربه‌فرد:**
   - با استفاده از داده‌های آموزش، لیستی از برچسب‌های احساسی استخراج و مرتب‌سازی شد.
   - این لیست برای نگاشت بین شناسه عددی و برچسب متنی در مرحله بعد استفاده شد.
4. **بارگذاری مدل و پردازشگر اولیه:(Feature Extractor)**
   - مدل پایه‌ی **facebook/wav2vec2-base** از کتابخانه‌ی HuggingFace بارگذاری شد.
   - تعداد برچسب‌ها (برای تنظیم خروجی مدل)، دیکشنری‌های نگاشت label2idو id2label، و نوع تسک (wav2vec2_clf) در پیکربندی مدل مشخص شدند.
   - همچنین پارامتر pooling_modeبه صورت دستی به تنظیمات مدل افزوده شد (در اینجا حالت میانگین).
5. **استخراج نرخ نمونه‌برداری:**

- نرخ نمونه‌برداری مورد انتظار مدل از طریق feature_extractorبه دست آمد که در مراحل پیش‌پردازش داده‌های صوتی استفاده خواهد شد.

اهمیت این مرحله:

- استفاده از مدل‌های **پیش‌آموخته (Pretrained)** مانند Wav2Vec2 باعث کاهش چشمگیر نیاز به داده‌های برچسب‌خورده می‌شود.
- تنظیم دقیق برچسب‌ها و ساختار مدل، شرط لازم برای fine-tuning موفق مدل روی داده‌های صوتی احساسات است.
- بارگذاری استاندارد داده‌ها از طریق datasets، امکان بهره‌گیری از ابزارهای پیشرفته‌ی HuggingFaceرا فراهم می‌سازد.

بخش چهارم (پیش‌پردازش داده‌های صوتی):

در این مرحله، داده‌های صوتی برای استفاده به عنوان ورودی مدل Wav2Vec2.0 پیش‌پردازش شدند. هدف از این مرحله، آماده‌سازی سیگنال‌های صوتی با نرخ نمونه‌برداری مناسب و تبدیل آن‌ها به فرمتی است که پردازشگر صوت مدل بتواند آن را تحلیل کند.

مراحل انجام‌شده:

1. **بارگذاری و بازنمونه‌برداری(Resampling)** :
   - با استفاده از کتابخانه‌ی librosa، فایل‌های صوتی از مسیر مشخص‌شده بارگذاری شدند.
   - تمامی صداها به نرخ نمونه‌برداری مورد انتظار مدل (که از Wav2Vec2Processorاستخراج شده بود) بازنمونه‌برداری شدند تا ورودی مدل هماهنگ باشد.

2. **استخراج ویژگی‌ها(Feature Extraction)** :

   - با استفاده از feature_extractorمدل Wav2Vec2 ، سیگنال‌های صوتی خام به ویژگی‌های عددی قابل‌فهم برای مدل تبدیل شدند. این ویژگی‌ها شامل بردارهای input_valuesهستند که مستقیماً به مدل داده می‌شوند.

3. **برچسب‌گذاری داده‌ها:**

   - برچسب متنی احساسات به عدد صحیح (کلاس) تبدیل شد تا به عنوان خروجی برای یادگیری مدل مورد استفاده قرار گیرد.

4. **اجرای پیش‌پردازش روی کل دیتاست:**

   - تمام مجموعه‌های آموزش، اعتبارسنجی و آزمون با استفاده از تابع mapاز کتابخانه‌ی HuggingFace datasetsپردازش شدند. این عملیات به صورت batch صورت نگرفت و روی هر نمونه به‌صورت جداگانه اعمال شد.

بخش پنجم (تعریف مدل طبقه‌بندی گفتار و آماده‌سازی داده‌ها برای آموزش):

در این مرحله، یک مدل سفارشی مبتنی بر Wav2Vec2 برای طبقه‌بندی احساسات از سیگنال صوتی طراحی شده و همچنین یک داده‌پرداز (Data Collator) تعریف گردید تا داده‌ها در هنگام آموزش به شکل مناسب به مدل داده شوند.

1. تعریف مدل Wav2Vec2ForSpeechClassification :

مدل طراحی‌شده از کلاس پایه‌ی Wav2Vec2PreTrainedModelارث‌بری شده و به شرح زیر است:

- **مدل پایه(Backbone)** : از مدل Wav2Vec2Model استفاده شده که سیگنال صوتی خام را به بردارهای ویژگی (feature embeddings) تبدیل می‌کند.
- **Pooling**: بر اساس تنظیمات، یکی از روش‌های mean, sum, یا max برای فشرده‌سازی ویژگی‌ها در طول زمان استفاده می‌شود. این کار برای تطابق با ورودی لایه‌ی fully connected ضروری است.

- **لایه‌های طبقه‌بندی:** از یک شبکه‌ی ساده شامل دو لایه‌ی Linear با توابع GELU و Dropout استفاده شده که ویژگی‌ها را به خروجی نهایی (برچسب احساس) نگاشت می‌کنند.
- **تابع خطا:** برای آموزش، تابع CrossEntropyLoss استفاده شده که مناسب برای مسائل چندکلاسه است.

توجه: خط کد مربوط به **غیرفعال‌سازی گرادیان‌ها در wav2vec2** (param.requires_grad = False) به صورت کامنت باقی مانده که در صورت استفاده به آموزش فقط لایه‌های بالایی محدود می‌شود (روش *feature extraction* به جای *fine-tuning*).

2. تعریف Data Collator :

در هنگام آموزش مدل، لازم است تا ورودی‌ها دارای طول یکسان باشند. از آن‌جایی که سیگنال‌های صوتی طول‌های متفاوتی دارند، از کلاس DataCollator برای **pad کردن داینامیک سیگنال‌ها** استفاده شده است:

- مقادیر input_values با استفاده از feature_extractor.pad هم‌طول شده‌اند.
- برچسب‌ها به صورت tensor جداگانه‌ای به مدل داده می‌شوند.

این ساختار باعث می‌شود تا در هر batch از آموزش، داده‌ها با ساختار مناسب و هماهنگ به مدل داده شوند.

بخش ششم (آموزش مدل، تنظیم Optimizer و Early Stopping):

پس از آماده‌سازی داده‌ها و تعریف مدل، فرآیند آموزش به همراه ارزیابی میان‌دوره‌ای و کنترل overfitting انجام شد.

1. آماده‌سازی DataLoader ها:

با استفاده از DataLoader، داده‌ها به صورت batch به مدل داده شدند. از داده‌پرداز تعریف‌شده (collator) برای هم‌طول کردن نمونه‌های صوتی استفاده شد. اندازه‌ی batch برابر با 16 انتخاب گردید.

2. انتخاب بهینه‌ساز و Scheduler:

- **بهینه‌ساز :**از AdamW با نرخ یادگیری اولیه 3e-5و ضریب افت وزن 0.01استفاده شد.
- **تنظیم نرخ یادگیری :**با استفاده از linear schedulerو warmup برابر با 10٪ از تعداد کل قدم‌های آموزش، نرخ یادگیری در طول آموزش به‌صورت خطی کاهش می‌یابد.

3. پیاده‌سازی Early Stopping:

به منظور جلوگیری ازoverfitting ، از توقف زودهنگام استفاده شد:

- اگر مدل برای 6 دوره‌ی متوالی نتواند عملکرد خود را در مجموعه‌ی اعتبارسنجی (validation) بهبود دهد، آموزش متوقف می‌شود.
- در هر دوره، اگر مدل عملکرد بهتری داشت، وزن‌های آن به عنوان "بهترین مدل" ذخیره می‌شوند.

4. حلقه‌ی آموزش:

مدل به مدت حداکثر 50 دوره آموزش داده شد. در هر دوره:

- مدل در حالت train قرار گرفت.
- پیش‌پردازش و محاسبات forward و backward انجام شد.
- از **gradient clipping**برای جلوگیری از انفجار گرادیان‌ها استفاده شد.
- پس از پایان هر دوره، عملکرد مدل روی مجموعه‌ی validation محاسبه شد.

- شاخص‌هایی چون **دقت (accuracy)** و **خطای میانگین (loss)** در هر دوره چاپ گردیدند.

در صورت عدم بهبود loss در validation ، شمارنده‌ی early stopping افزایش یافته و در نهایت آموزش متوقف می‌شود.

```
Epoch 1: 100%|██████████| 533/533 [14:44<00:00,  1.66s/it]
Epoch 1 | Train Loss: 1.6903 | Train Acc: 0.3800 | Val Loss: 1.2171 | Val Acc: 0.6425
Epoch 2: 100%|██████████| 533/533 [14:38<00:00,  1.65s/it]
Epoch 2 | Train Loss: 0.9775 | Train Acc: 0.6981 | Val Loss: 0.8903 | Val Acc: 0.7122
Epoch 3: 100%|██████████| 533/533 [14:39<00:00,  1.65s/it]
Epoch 3 | Train Loss: 0.6923 | Train Acc: 0.7671 | Val Loss: 0.7580 | Val Acc: 0.7516
Epoch 4: 100%|██████████| 533/533 [14:39<00:00,  1.65s/it]
Epoch 4 | Train Loss: 0.4999 | Train Acc: 0.8328 | Val Loss: 0.6432 | Val Acc: 0.7950
Epoch 5: 100%|██████████| 533/533 [14:46<00:00,  1.66s/it]
Epoch 5 | Train Loss: 0.3952 | Train Acc: 0.8707 | Val Loss: 0.5867 | Val Acc: 0.8158
Epoch 6: 100%|██████████| 533/533 [14:46<00:00,  1.66s/it]
Epoch 6 | Train Loss: 0.3176 | Train Acc: 0.9006 | Val Loss: 0.7067 | Val Acc: 0.8004
Epoch 7: 100%|██████████| 533/533 [14:43<00:00,  1.66s/it]
Epoch 7 | Train Loss: 0.2593 | Train Acc: 0.9202 | Val Loss: 0.8611 | Val Acc: 0.7982
Epoch 8: 100%|██████████| 533/533 [14:45<00:00,  1.66s/it]
Epoch 8 | Train Loss: 0.2181 | Train Acc: 0.9399 | Val Loss: 0.8472 | Val Acc: 0.8125
Epoch 9: 100%|██████████| 533/533 [14:43<00:00,  1.66s/it]
Epoch 9 | Train Loss: 0.1795 | Train Acc: 0.9558 | Val Loss: 0.8912 | Val Acc: 0.8120
Epoch 10: 100%|██████████| 533/533 [14:45<00:00,  1.66s/it]
Epoch 10 | Train Loss: 0.1427 | Train Acc: 0.9661 | Val Loss: 1.0040 | Val Acc: 0.8169
Epoch 11: 100%|██████████| 533/533 [14:44<00:00,  1.66s/it]
Epoch 11 | Train Loss: 0.1225 | Train Acc: 0.9715 | Val Loss: 1.1703 | Val Acc: 0.8114
 Early stopping triggered.
```

همان طور که مشاهده میشود بخاطر افزایش Val Loss از epoch 10 به 11، فرآیند Early stopping صورت گرفته و آموزش متوقف شده است.

بخش هفتم (ارزیابی نهایی روی مجموعه تست):

پس از پایان آموزش مدل و اعمال Early Stopping ، ارزیابی نهایی روی مجموعه‌ی تست انجام شد تا میزان **عمومی‌سازی (generalization)** مدل بررسی گردد.

1. معیارهای ارزیابی:

در این مرحله از معیارهای مختلفی برای بررسی عملکرد مدل استفاده شده است:

- **Accuracy(دقت کلی):** نسبت پیش‌بینی‌های صحیح به کل نمونه‌ها.
- **F1-Score(میانگین وزنی و ماکرو):**
  - **Macro F1:** میانگین F1 برای همه کلاس‌ها، بدون توجه به فراوانی آن‌ها.
  - **Weighted F1:** میانگین F1 با درنظر گرفتن فراوانی کلاس‌ها.

2. گزارش دسته‌بندی(Classification Report) :

با استفاده از classification_report، معیارهای دقیق مانندprecision ، recallو F1 برای هر کلاس به صورت مجزا نمایش داده شد. این گزارش کمک می‌کند تا نقاط ضعف مدل در تشخیص برخی احساسات مشخص شود.

3. ماتریس سردرگمی(Confusion Matrix) :

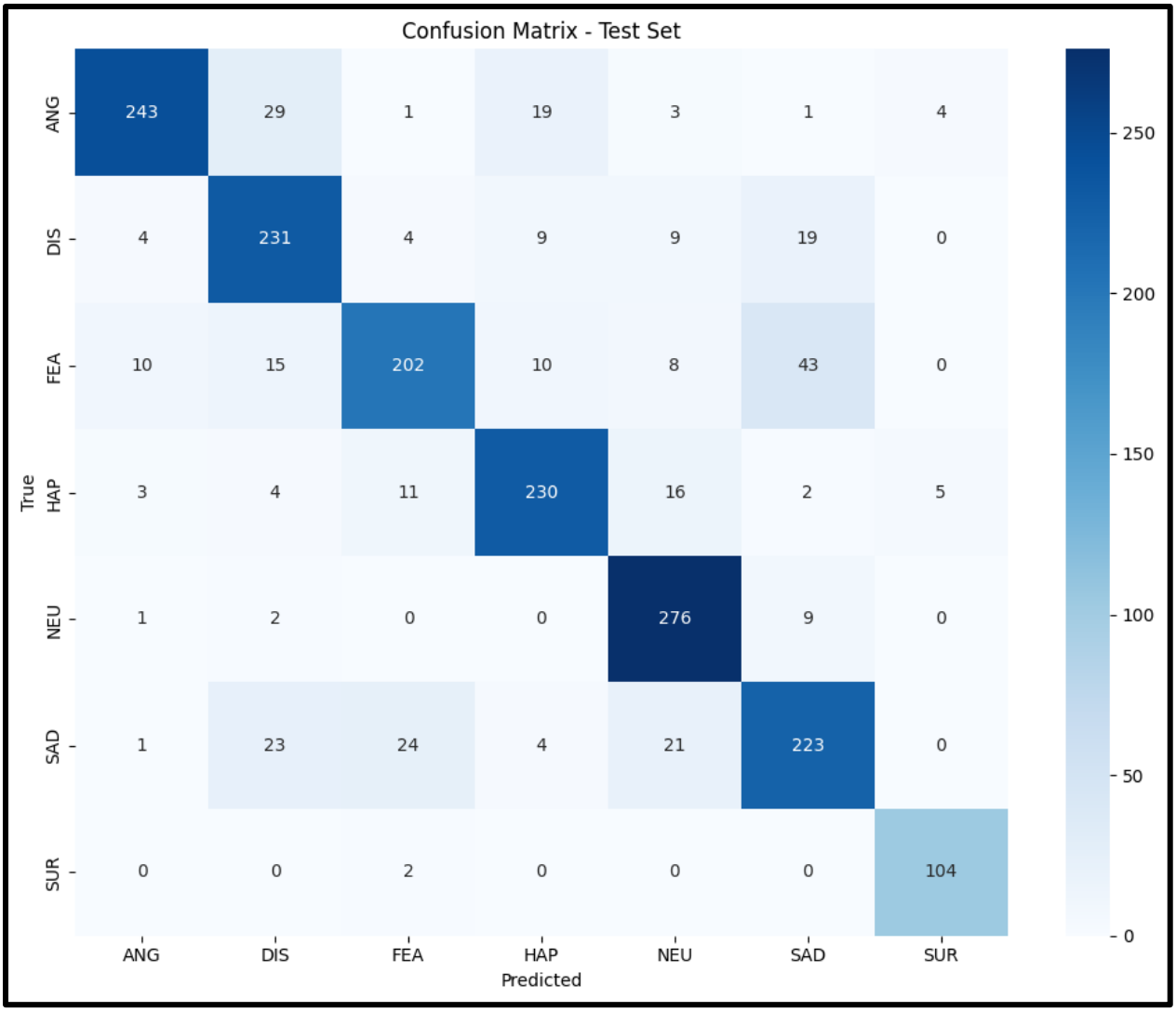
ماتریس سردرگمی با استفاده از seaborn ترسیم شد تا بصری‌سازی واضحی از اشتباهات مدل فراهم شود.

- سطرها نشان‌دهنده‌ی **احساسات واقعی** هستند.
- ستون‌ها نشان‌دهنده‌ی **پیش‌بینی مدل** هستند.
- خانه‌های قطر اصلی نشان‌دهنده‌ی تعداد پیش‌بینی صحیح در هر کلاس‌اند.

```
Classification Report:
              precision    recall  f1-score   support

         ANG       0.93      0.81      0.86       300
         DIS       0.76      0.84      0.80       276
         FEA       0.83      0.70      0.76       288
         HAP       0.85      0.85      0.85       271
         NEU       0.83      0.96      0.89       288
         SAD       0.75      0.75      0.75       296
         SUR       0.92      0.98      0.95       106

    accuracy                           0.83      1825
   macro avg       0.84      0.84      0.84      1825
weighted avg       0.83      0.83      0.83      1825
```



- دقت کلی (Accuracy): 83٪
- F1-Scoreمیانگین ماکرو → 84٪ :نشان‌دهنده تعادل مناسب در عملکرد برای همه کلاس‌هاست.
- F1-Scoreمیانگین وزن‌دار → 83٪ :عملکرد مدل به خوبی در داده‌های نامتوازن هم حفظ شده.

نکات مثبت:

- بهترین عملکرد مربوط به کلاس شگفتی (SUR) با F1=0.95 است، یعنی مدل این احساس را بسیار خوب تشخیص داده.
- کلاس‌های خنثی (NEU) و خشم (ANG) هم عملکرد بالایی دارند.
- مدل در طبقه‌بندی شادی (HAP) نیز بسیار دقیق بوده.

نقاط قابل بهبود (بر اساس ماتریس سردرگمی)

- کلاس ترس (FEA) به دفعات با غم (SAD) اشتباه گرفته شده (43 مورد اشتباه). این دو احساس ممکنه از نظر صوتی شباهت‌هایی داشته باشن.
- غم (SAD) به‌طور قابل توجهی با انزجار (DIS) و ترس (FEA) اشتباه گرفته شده (مجموعاً 47 مورد).
- خشم (ANG) با انزجار (DIS) و شادی (HAP) بعضاً اشتباه شده، که ممکنه به نحوه بیان مشابه در شدت صدا برگرده.

متن:

بخش اول:

هدف این پروژه، توسعه یک مدل یادگیری عمیق مبتنی بر ترانسفورمر برای طبقه‌بندی احساسات موجود در توییت‌ها به دسته‌های مختلف (مانند خشم، شادی، عشق، خنثی، تعجب و نگرانی) است. داده‌های استفاده‌شده از مجموعه داده‌ای به نام tweet_emotions.csvاستخراج شده‌اند که شامل توییت‌هایی با برچسب‌های احساسی است. این نوت‌بوک شامل مراحل مختلفی از جمله تحلیل اکتشافی داده‌ها(EDA) ، پیش‌پردازش متن، ایجاد تعبیه‌هایGloVe ، ساخت و آموزش مدل ترانسفورمر، و در نهایت پیاده‌سازی یک ربات تلگرامی برای تعامل با کاربران است.

در این گزارش، هر سلول کد به‌صورت جداگانه بررسی شده و توضیح داده می‌شود. ابتدا عملکرد کد توضیح داده می‌شود، سپس نتایج آن تفسیر می‌گردد، و در صورت لزوم پیشنهادهایی برای بهبود ارائه خواهد شد. ا

در این قسمت میخواهیم احساس داده های متنی را استخراج کنیم برای اینکار ابتدا :

```
df = pd.read_csv('/kaggle/input/emotion-detection-from-text/tweet_emotions.csv', delimiter=',')
df.head()
```

توضیح :این سلول داده‌ها را از فایل tweet_emotions.csvبارگذاری می‌کند و 5 ردیف اول را نمایش می‌دهد. داده‌ها شامل سه ستون هستند:

- **tweet_id:شناسه منحصربه‌فرد توییت.**
- **sentiment:برچسب احساسی (مثل خنثی، غم، شادی).**
- **content:متن توییت.**

	tweet_id	sentiment	content
0	1956967341	empty	@tiffanylue i know i was listenin to bad habi...
1	1956967666	sadness	Layin n bed with a headache ughhhh...waitin o...
2	1956967696	sadness	Funeral ceremony...gloomy friday...
3	1956967789	enthusiasm	wants to hang out with friends SOON!
4	1956968416	neutral	@dannycastillo We want to trade with someone w...

- داده‌ها شامل توییت‌هایی با احساسات متنوع (مثل خالی، غم، اشتیاق، خنثی) هستند.
- ستون contentحاوی متن خام توییت‌هاست که نیاز به پیش‌پردازش (مثل حذف کاراکترهای خاص یا نرمال‌سازی) دارد.
- وجود برچسب‌های احساسی متنوع نشان‌دهنده یک مسئله طبقه‌بندی چندکلاسه است.

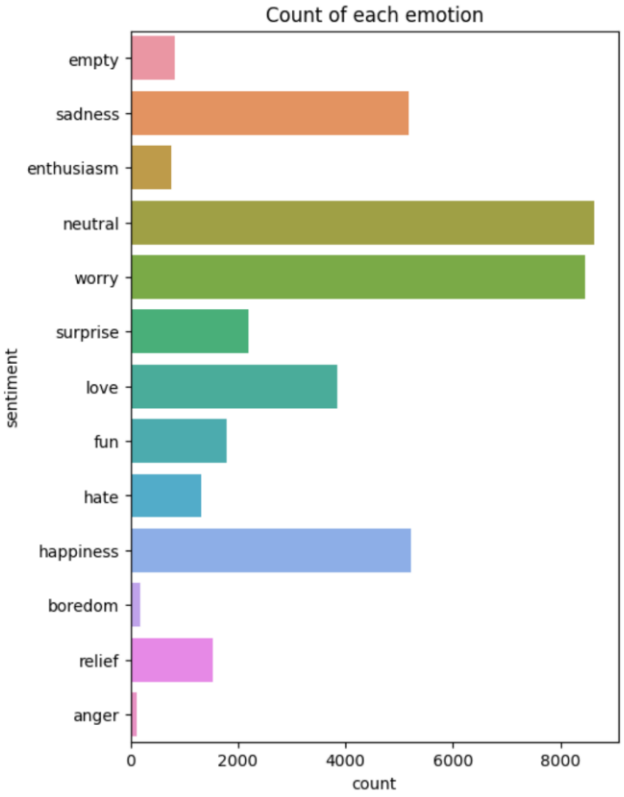
سلول ۳ :

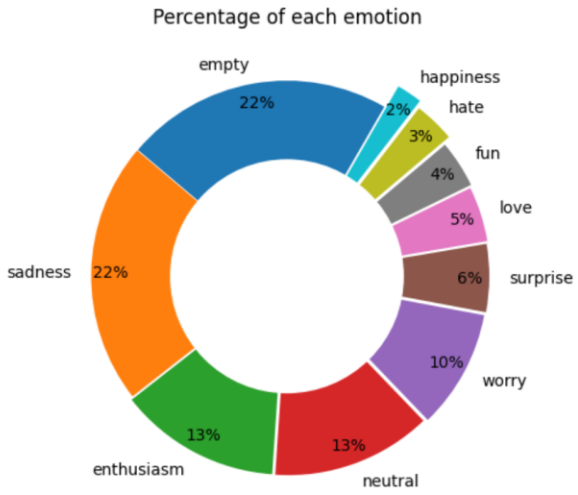
تعریف تابع تحلیل اکتشافی داده‌ها(EDA)

توضیح :این سلول یک تابع به نام basic_edaتعریف می‌کند که تحلیل اکتشافی داده‌ها را انجام می‌دهد. این تابع:

- تعداد ستون‌ها و ردیف‌های داده‌ها را چاپ می‌کند.
- نوع داده‌های هر ستون (عددی یا دسته‌ای) را بررسی کرده و تعداد ستون‌های هر نوع را گزارش می‌دهد.
- نام ستون‌های دسته‌ای و عددی را نمایش می‌دهد.
- اطلاعات آماری ستون‌های عددی (مثل میانگین، انحراف معیار) و دسته‌ای (مثل تعداد، تعداد مقادیر یکتا) را ارائه می‌دهد.

توضیح خلاصه ای از تعداد داده ها و توزیع آنها :





باتوجه به زیاد بودن دسته ها اقدام به ادغام بعضی از آنها باهم می کنیم و نهایتا به ۶ دسته میرسیم :

مشاهدات :

- برچسب emptyبه neutral، sadnessبه worry، enthusiasmبه surprise، و غیره نگاشت شده‌اند.
- این نگاشت باعث ساده‌سازی مسئله طبقه‌بندی از 13 کلاس به 6 کلاس می‌شود، که می‌تواند دقت مدل را بهبود دهد، به‌ویژه با توجه به توزیع نامتوازن برچسب‌ها (همان‌طور که در سلول‌های قبلی مشاهده شد).
- برای مثال، توییت مربوط به سردرد (ردیف 1) اکنون به‌درستی به‌عنوان worryبرچسب‌گذاری شده است، که با محتوای آن سازگار است

تفسیر :

- کاهش تعداد کلاس‌ها یک استراتژی منطقی برای ساده‌سازی مسئله است، به‌ویژه زمانی که برخی برچسب‌ها (مثل boredomیا reliefممکن است تعداد کمی نمونه داشته باشند.
- ادغام برچسب‌های مشابه (مثل funو happinessبه happinessبه مدل کمک می‌کند تا الگوهای قوی‌تری یاد بگیرد.
- با این حال، این نگاشت ممکن است برخی تفاوت‌های ظریف بین احساسات را از بین ببرد (مثلاً emptyو neutralممکن است در برخی زمینه‌ها معانی متفاوتی داشته باشند.(

تابع text_preprocessing_platform :

این سلول یک تابع جامع به نام text_preprocessing_platformتعریف می‌کند که برای پیش‌پردازش متون طراحی شده است. این تابع مراحل مختلفی را برای پاک‌سازی و نرمال‌سازی متون انجام می‌دهد تا برای مدل‌سازی مناسب شوند. مراحل و اجزای اصلی تابع عبارتند از:

1. **پاک‌سازی اولیه :(denoise_text)**
   - حذف تگ‌های) HTML با استفاده از (BeautifulSoupدر صورت وجود.
   - رفع انقباض‌ها) مثلاً تبدیل "didn't" به ("did not" با استفاده از کتابخانه .contractions
2. **نرمال‌سازی متن :(normalize_text)**
   - **حذف کاراکترهای غیر :ASCII** برای حذف کاراکترهای خاص یا غیرانگلیسی (مثل اموجی‌ها).
   - **تبدیل به حروف کوچک** :برای یکنواخت کردن متن.
   - **حذف علائم نگارشی** :با استفاده از regex برای حذف کاراکترهایی مثل نقطه، کاما، و غیره.
   - **جایگزینی اعداد با معادل متنی** :تبدیل اعداد به کلمات) مثلاً "2" به ("two" با استفاده از کتابخانه .inflect
   - **حذف کلمات توقف :(Stopwords)** حذف کلمات رایج و کم‌اهمیت) مثل"the" ، ("is"با استفاده از لیست کلمات توقف) NLTK در صورت فعال بودن گزینه remove_stopwords).

- **ریشه‌یابی :(Stemming)** این بخش غیرفعال شده (با کامنت) و استفاده نشده است.
- **لماتیزه کردن افعال** :تبدیل افعال به شکل پایه) مثلاً "running" به ("run" با استفاده از WordNetLemmatizer.

3. **توکن‌سازی :(tokenize)**
   - با استفاده از nltk.word_tokenize، متن به کلمات جداگانه تقسیم می‌شود.
4. **تابع اصلی :(text_prepare)**
   - ترکیب مراحل پاک‌سازی و نرمال‌سازی برای پردازش هر متن.
   - خروجی به‌صورت یک رشته متنی است که کلمات نرمال‌شده با فاصله از هم جدا شده‌اند.
5. **اعمال روی :DataFrame**
   - تابع روی ستون متنی مشخص‌شده (text_col) از DataFrame اعمال می‌شود و متن‌های پردازش‌شده جایگزین متن‌های اصلی می‌شوند.

نتایج و تفسیر : این سلول خروجی خاصی تولید نمی‌کند، زیرا فقط تابع را تعریف می‌کند. با این حال، هدف این تابع آماده‌سازی متون برای مدل‌سازی است. ویژگی‌های کلیدی و نکات قابل‌توجه عبارتند از:

- **جامعیت تابع :**این تابع مجموعه‌ای از مراحل استاندارد پیش‌پردازش متن را پوشش می‌دهد که برای تحلیل احساسات بسیار مناسب است.
- **انعطاف‌پذیری :**گزینه remove_stopwordsامکان کنترل حذف یا حفظ کلمات توقف را فراهم می‌کند، که در برخی موارد (مثلاً وقتی کلمات توقف برای تحلیل احساسات مهم هستند) مفید است.
- **لماتیزه کردن به‌جای ریشه‌یابی :**استفاده از لماتیزه کردن به‌جای ریشه‌یابی (که غیرفعال شده) انتخاب خوبی است، زیرا لماتیزه کردن شکل معنادار کلمات را حفظ می‌کند) مثلاً "better" به "good" تبدیل می‌شود، نه یک ریشه غیرواقعی.(
- **پشتیبانی از متن‌های غیراستاندارد :**حذف کاراکترهای غیر ASCII و رفع انقباض‌ها نشان‌دهنده توجه به متون غیررسمی توییت‌هاست که ممکن است شامل اموجی یا انقباض‌های زبانی باشند.

نتایج :

Before Text Preprocessing

	content
0	@tiffanylue i know i was listenin to bad habi...
1	Layin n bed with a headache ughhhh...waitin o...
2	Funeral ceremony...gloomy friday...
3	wants to hang out with friends SOON!
4	@dannycastillo We want to trade with someone w...

After Text Preprocessing

	content
0	tiffanylue know listenin bad habit earlier sta...
1	layin n bed headache ughhhh waitin call
2	funeral ceremony gloomy friday
3	want hang friends soon
4	dannycastillo want trade someone houston ticke...

تعریف مدل :

توضیح :این سلول شامل دو بخش اصلی است:

1. **تنظیم لاگینگ:**

- با استفاده از logging.basicConfig، سیستم لاگینگ پیکربندی می‌شود تا پیام‌های سطح INFOو بالاتر با فرمت مشخص (شامل زمان، سطح پیام، و متن پیام) ثبت شوند. این برای ردیابی فرآیندهای آموزش و اشکال‌زدایی مفید است.

2. **تعریف مدل ترانسفورمر:**

- دو کلاس PyTorch تعریف شده‌اند PositionalEncoding :و TransformerTextClassifier.

الف. کلاس :PositionalEncoding

- این کلاس برای افزودن اطلاعات موقعیتی به تعبیه‌های کلمات در مدل ترانسفورمر استفاده می‌شود، زیرا ترانسفورمرها به‌طور ذاتی ترتیب کلمات را در نظر نمی‌گیرند.

- از توابع سینوسی و کسینوسی برای ایجاد بردارهای موقعیتی استفاده می‌کند که به ورودی اضافه می‌شوند.
- پارامترها :
  - d_model: ابعاد تعبیه‌ها (50 در اینجا).
  - dropout: نرخ Dropout برای جلوگیری از بیش‌برازش.(0.1)
  - max_len: حداکثر طول توالی.(5000)
- خروجی: ورودی با اطلاعات موقعیتی و اعمال Dropout.

ب. کلاس :TransformerTextClassifier

- این کلاس مدل اصلی ترانسفورمر برای طبقه‌بندی متن است. اجزای آن عبارتند از :

- **لایه تعبیه :(Embedding)** از تعبیه‌های GloVe از پیش‌آموزش‌دیده (50-بعدی) استفاده می‌کند. ماتریس تعبیه با استفاده از embeddings_index و word_index مقداردهی اولیه می‌شود و وزن‌های آن ثابت نگه داشته می‌شوند (requires_grad = False).
- **رمزگذار موقعیتی :(PositionalEncoding)** برای افزودن اطلاعات ترتیب کلمات.
- **رمزگذار ترانسفورمر :(TransformerEncoder)** شامل num_layers(2 لایه) از لایه‌های ترانسفورمر با num_heads(5 سر توجه) و یک لایه خوراک پیش‌رو با 256 واحد.
- **لایه‌های کاملاً متصل :(fc1, fc2)** برای نگاشت خروجی ترانسفورمر به تعداد کلاس‌ها (6 کلاس). لایه اول به 25 واحد کاهش می‌یابد، و لایه دوم به تعداد کلاس‌ها.
- **Dropout و :ReLU** برای تنظیم مدل و افزودن غیرخطی بودن.

- **پارامترهای ورودی :**
  - word_index: دیکشنری واژگان توکنایزر.

- embeddings_index: تعبیه‌های GloVe.
- nclasses: تعداد کلاس‌ها (6).
- max_seq_len: حداکثر طول توالی (500).
- embedding_dim: ابعاد تعبیه (50).
- dropout: نرخ Dropout (0.2).
- num_heads: تعداد سرهای توجه (5).
- num_layers: تعداد لایه‌های ترانسفورمر (2).

- **جریان پیش‌رو (forward):**

- ورودی توکن‌شده به تعبیه تبدیل می‌شود و با ضریب sqrt(50) مقیاس‌بندی می‌شود.
- اطلاعات موقعیتی اضافه می‌شود.
- از رمزگذار ترانسفورمر عبور می‌کند.
- میانگین توالی گرفته می‌شود (mean(dim=1)) تا یک بردار برای هر نمونه تولید شود.
- از لایه‌های کاملاً متصل و فعال‌سازی ReLU عبور می‌کند و خروجی نهایی (احتمال کلاس‌ها) تولید می‌شود.

نتایج و تفسیر : این سلول خروجی خاصی تولید نمی‌کند، زیرا صرفاً لاگینگ را تنظیم کرده و کلاس‌های مدل را تعریف می‌کند. با این حال، تحلیل ساختار مدل و پیکربندی آن نکات زیر را نشان می‌دهد:

- **انتخاب ترانسفورمر :** استفاده از معماری ترانسفورمر برای تحلیل احساسات انتخابی پیشرفته است، زیرا این مدل‌ها در درک روابط پیچیده بین کلمات (به‌ویژه در توییت‌های کوتاه) قوی هستند.
- **تعبیه‌های GloVe:** استفاده از تعبیه‌های از پیش‌آموزش‌دیده GloVe 50-) بعدی (باعث کاهش زمان آموزش و بهبود درک معنایی کلمات می‌شود. ثابت نگه داشتن وزن‌های تعبیه (requires_grad = False) باعث کاهش پیچیدگی محاسباتی می‌شود، اما ممکن است انعطاف‌پذیری مدل را محدود کند.

- **پیکربندی مدل :**
  - تعداد 5 سر توجه و 2 لایه ترانسفورمر برای داده‌های متنی کوتاه (توییت‌ها) معقول است، اما ممکن است برای داده‌های پیچیده‌تر نیاز به تنظیم داشته باشد.
  - نرخ Dropout (0.2) و لایه خوراک پیش‌رو با 256 واحد نشان‌دهنده تلاش برای جلوگیری از بیش‌برازش است.
  - کاهش ابعاد به 25 در لایه اول کاملاً متصل ممکن است اطلاعات را فشرده کند، که باید از نظر تأثیر روی دقت بررسی شود.

توضیح :این سلول چهار تابع کلیدی برای پیش‌پردازش داده‌ها، بارگذاری تعبیه‌ها، طبقه‌بندی متن، و تبدیل صوت به متن تعریف می‌کند. در ادامه، هر تابع به‌صورت جداگانه بررسی می‌شود:

1. **تابع :load_glove_embeddings**
   - **هدف :**بارگذاری تعبیه‌های GloVe از یک فایل متنی.
   - **عملکرد :**فایل) GloVe در اینجا (glove.6B.50d.txtرا خط به خط می‌خواند. هر خط شامل یک کلمه و بردار تعبیه 50-بعدی آن است. کلمات و بردارها در یک دیکشنری (embeddings_index)ذخیره می‌شوند.
   - **ورودی :**مسیر فایل.GloVe
   - **خروجی :**دیکشنری که کلیدها کلمات و مقادیر بردارهای تعبیه هستند.
2. **تابع :loadData_Tokenizer**
   - **هدف :**توکن‌سازی و آماده‌سازی داده‌های آموزشی و آزمایشی برای مدل.
   - **عملکرد :**
     - یک شیء Tokenizerاز Keras ایجاد می‌کند و روی داده‌های آموزشی (X_train) آموزش می‌دهد تا واژگان را بسازد.(word_index)

- متون آموزشی و آزمایشی را به دنباله‌های عددی تبدیل می‌کند (texts_to_sequences).
- دنباله‌ها را با طول ثابت (max_seq_len=500) پد می‌کند (pad_sequences)تا ورودی‌های مدل یکنواخت باشند.
- تعبیه‌های GloVe را با استفاده از تابع load_glove_embeddingsبارگذاری می‌کند.

- **ورودی :**داده‌های آموزشی و آزمایشی، مسیر فایلGloVe ، و حداکثر طول توالی.
- **خروجی :**داده‌های پدشده آموزشی و آزمایشی، دیکشنری واژگان، تعبیه‌هایGloVe ، و شیء توکنایزر.

3. **تابع :classify_text**

- **هدف :**طبقه‌بندی احساس یک متن ورودی با استفاده از مدل آموزش‌دیده.
- **عملکرد :**
  - متن ورودی را با استفاده از توکنایزر به دنباله عددی تبدیل می‌کند.
  - دنباله را پد می‌کند تا با طول ثابت (max_seq_len) مطابقت داشته باشد.
  - دنباله را به تنسور PyTorch تبدیل کرده و به دستگاه مناسب (CPU یا GPU) منتقل می‌کند.
  - مدل را در حالت ارزیابی (eval) قرار داده و پیش‌بینی را انجام می‌دهد.
  - کلاس با بالاترین احتمال (argmax) به‌عنوان خروجی بازمی‌گردد.
- **ورودی :**متن، مدل، توکنایزر، حداکثر طول توالی، و دستگاه.
- **خروجی :**شماره کلاس پیش‌بینی‌شده.

نتایج و تفسیر :این سلول خروجی خاصی تولید نمی‌کند، زیرا فقط توابع را تعریف می‌کند. با این حال، تحلیل عملکرد و طراحی توابع نکات زیر را نشان می‌دهد:

- **بارگذاری GloVe:** تابع load_glove_embeddingsبه‌خوبی برای بارگذاری تعبیه‌های 50-بعدی GloVeطراحی شده است. این تعبیه‌ها اطلاعات معنایی غنی‌ای را برای مدل فراهم می‌کنند.
- **توکن‌سازی :**تابع loadData_Tokenizerمراحل استاندارد توکن‌سازی و پد کردن را به‌خوبی انجام می‌دهد. استفاده از max_seq_len=500برای توییت‌ها ممکن است بیش از حد باشد، زیرا توییت‌ها معمولاً کوتاه‌تر هستند (حداکثر 280 کاراکتر).
- **طبقه‌بندی متن :**تابع classify_textساده و کارآمد است و برای پیش‌بینی‌های بلادرنگ (مثل ربات تلگرامی) مناسب طراحی شده است. استفاده از torch.no_grad()مصرف حافظه را در زمان استنتاج کاهش می‌دهد.
- **تبدیل صوت به متن :**تابع voice_to_textقابلیت جذابی برای پشتیبانی از ورودی‌های صوتی اضافه می‌کند، که برای ربات تلگرامی بسیار کاربردی است. با این حال، وابستگی به سرویس Google Speech Recognitionممکن است در محیط‌های آفلاین یا با محدودیت دسترسی مشکل‌ساز باشد.
- **مدیریت خطاها :**تابع voice_to_textمدیریت خطای خوبی دارد (برای خطاهای تشخیص گفتار و دسترسی به سرویس)، اما تابع classify_textمی‌تواند با بررسی ورودی‌های نامعتبر (مثل متن خالی) مقاوم‌تر شود.

در ادامه با توجه به categorical بودن برچسب ها آن ها را به عدد تبدیل می کنیم :

```
# Label encoding target column
le = LabelEncoder()
df['sentiment'] = le.fit_transform(df['sentiment'])
df.head(40)
```

سلول 10: آماده‌سازی داده‌ها و ایجاد مدل ترانسفورمر

توضیح :این سلول شامل مراحل آماده‌سازی داده‌ها برای آموزش مدل و ایجاد نمونه‌ای از مدل ترانسفورمر است. در ادامه، هر بخش به‌صورت جداگانه بررسی می‌شود:

1. **وارد کردن کتابخانه و تعریف متغیرها :**
   - **کتابخانه pickleبرای ذخیره توکنایزر استفاده می‌شود.**
   - **Xبه ستون content(متن توییت‌ها) و yبه ستون sentiment(برچسب‌های احساسی) از DataFrameتخصیص داده می‌شود.**
   - **مسیر فایل تعبیه‌های GloVe (glove.6B.50d.txt) در glove_pathذخیره می‌شود.**
2. **تقسیم داده‌ها :**
   - **با استفاده از train_test_splitاز کتابخانهScikit-learn ، داده‌ها به دو مجموعه آموزشی (80٪) و آزمایشی (20٪) تقسیم می‌شوند.**
   - **خروجی شامل X_train, X_test, y_train, y_testاست.**
3. **توکن‌سازی و بارگذاری تعبیه‌ها :**
   - **تابع loadData_Tokenizer(تعریف‌شده در سلول قبلی) فراخوانی می‌شود تا :**
     - **متون آموزشی و آزمایشی را توکن‌سازی کند.**
     - **دنباله‌های عددی را با طول ثابت 500 پد کند (X_train_Glove, X_test_Glove).**
     - **دیکشنری واژگان (word_index) و تعبیه‌های GloVe (embeddings_index)را تولید کند.**
     - **شیء توکنایزر را برگرداند.**

- پیام "Generating Glove Embeddings..."چاپ می‌شود تا کاربر از پیشرفت فرآیند مطلع شود.

4. ذخیره توکنایزر :

- توکنایزر با استفاده از pickleدر فایل tokenizer.pklذخیره می‌شود تا در آینده (مثلاً برای ربات تلگرامی) قابل استفاده باشد.

5. تبدیل داده‌ها به تنسور :

- داده‌های پدشده (X_train_Glove, X_test_Glove) و برچسب‌ها (y_train, y_test) به تنسورهای PyTorch با نوع داده longتبدیل می‌شوند تا برای آموزش مدل ترانسفورمر آماده باشند.
- توجه: تبدیل y_trainو y_testبه آرایه NumPy قبل از ایجاد تنسور ممکن است به دلیل نوع داده‌های برچسب‌ها (مثلاً رشته) لازم باشد.

6. ایجاد مدل ترانسفورمر :

- یک نمونه از کلاس TransformerTextClassifier(تعریف‌شده در سلول قبلی) با پارامترهای زیر ایجاد می‌شود :
  - word_index:دیکشنری واژگان از توکنایزر.
  - embeddings_index:تعبیه‌های GloVe.
  - nclasses=6:تعداد کلاس‌های احساسی (خشم، شادی، عشق، خنثی، تعجب، نگرانی).
  - max_seq_len=500:حداکثر طول توالی.
  - embedding_dim=50:ابعاد تعبیه‌های GloVe.
  - dropout=0.2:نرخ Dropout برای تنظیم.

- num_heads=10: تعداد سرهای توجه در ترانسفورمر (متفاوت از سلول قبلی که 5 بود).
- num_layers=2: تعداد لایه‌های رمزگذار ترانسفورمر.

نتایج و تفسیر:

- تحلیل فرآیند :
  - تقسیم داده‌ها :تقسیم 80-20 استاندارد است و برای ارزیابی مدل مناسب به نظر می‌رسد. با 40,000 نمونه، حدود 32,000 نمونه برای آموزش و 8,000 نمونه برای آزمایش استفاده می‌شود.
  - توکن‌سازی و پد کردن :استفاده از طول توالی 500 ممکن است برای توییت‌ها بیش از حد باشد، زیرا توییت‌ها معمولاً کوتاه هستند (حداکثر 280 کاراکتر، معادل حدود 50-70 کلمه). این می‌تواند منجر به هدر رفتن حافظه و محاسبات برای پد کردن اضافی شود.
  - ذخیره توکنایزر :ذخیره توکنایزر با pickle برای استفاده‌های بعدی (مثل استنتاج در ربات تلگرامی) بسیار کاربردی است و از نیاز به بازسازی واژگان جلوگیری می‌کند.
  - تبدیل به تنسور :تبدیل داده‌ها به تنسورهای PyTorch با نوع long برای ورودی‌های مدل (توکن‌ها و برچسب‌ها) مناسب است. با این حال، فرض شده که برچسب‌ها (y_train, y_test) به‌صورت عددی هستند (0 تا 5)، که نیاز به بررسی دارد (در صورت رشته‌ای بودن، باید از LabelEncoder استفاده شود.)
  - ایجاد مدل :
    - استفاده از 10 سر توجه (به‌جای 5 در تعریف کلاس) نشان‌دهنده تلاش برای افزایش ظرفیت مدل در درک روابط پیچیده بین کلمات است.
    - مدل با 2 لایه ترانسفورمر و تعبیه‌های 50-بعدی نسبتاً سبک است، که برای داده‌های متنی کوتاه مثل توییت‌ها مناسب به نظر می‌رسد.

- **ثابت نگه داشتن تعبیه‌های GloVe) همان‌طور که در کلاس TransformerTextClassifierمشخص شده) باعث کاهش زمان آموزش می‌شود، اما ممکن است برای توییت‌های خاص (مثلاً با زبان غیررسمی یا اموجی) بهینه نباشد.**

در نهایت بعد از ترین کردن مدل به نتایج زیر رسیدیم :

[32]:
```
train_model(model , X_train_tensor , y_train_tensor , X_test_tensor , y_test_tensor)
```

```
Epoch 1: Train Loss = 1.6338, Train Acc = 0.3333 | Val Loss = 1.6082, Val Acc = 0.3379
✓ Saved new best model at epoch 1 (Val Loss: 1.6082)

Epoch 2: Train Loss = 1.6192, Train Acc = 0.3405 | Val Loss = 1.6083, Val Acc = 0.3379

Epoch 3: Train Loss = 1.6142, Train Acc = 0.3406 | Val Loss = 1.6074, Val Acc = 0.3379
✓ Saved new best model at epoch 3 (Val Loss: 1.6074)

Epoch 4: Train Loss = 1.6138, Train Acc = 0.3411 | Val Loss = 1.6069, Val Acc = 0.3379
✓ Saved new best model at epoch 4 (Val Loss: 1.6069)

Epoch 5: Train Loss = 1.6110, Train Acc = 0.3411 | Val Loss = 1.6081, Val Acc = 0.3379

Epoch 6: Train Loss = 1.6099, Train Acc = 0.3413 | Val Loss = 1.6068, Val Acc = 0.3379
✓ Saved new best model at epoch 6 (Val Loss: 1.6068)

---------------------------------------------------------------------------
KeyboardInterrupt                         Traceback (most recent call last)
/tmp/ipykernel_277/3914201083.py in <cell line: 0>()
----> 1 train_model(model , X_train_tensor , y_train_tensor , X_test_tensor , y_test_tensor)

/tmp/ipykernel_277/2791145692.py in train_model(model, X_train_tensor, y_train_tensor, X_test_tensor, y_test_tensor, batch_size, epochs, lr, weight_decay, patience, step_size, gamma, save_path)
     35             optimizer.step()
     36
---> 37             train_loss += loss.item()
     38             all_preds.extend(torch.argmax(outputs, dim=1).cpu().numpy())
     39             all_labels.extend(labels.cpu().numpy())

KeyboardInterrupt:
```

سلول 11: پیاده‌سازی ربات تلگرامی برای طبقه‌بندی احساسات

1. **اه‌اندازی ربات تلگرامی :**

- با استفاده از کتابخانه python-telegram-bot، یک شیء Updaterبا توکن تلگرام ایجاد می‌شود.
- سه مدیریت‌کننده (Handler) برای پردازش ورودی‌های کاربر تعریف شده‌اند :

- **دستور :/start**پیامی خوش‌آمدگویی ارسال می‌کند که کاربر را به ارسال متن یا پیام صوتی دعوت می‌کند.
- **پیام‌های متنی :**متن ورودی را با تابع classify_textپردازش کرده و احساس پیش‌بینی‌شده را نمایش می‌دهد.

- **پیام‌های صوتی :**پیام صوتی را دانلود کرده، با تابع voice_to_textبه متن تبدیل می‌کند، و سپس احساس متن را پیش‌بینی می‌کند. متن تبدیل‌شده و احساس پیش‌بینی‌شده به کاربر نمایش داده می‌شود. فایل صوتی موقت پس از پردازش حذف می‌شود.

- مدیریت‌کننده‌ها به Dispatcherاضافه شده و ربات با start_pollingشروع به کار می‌کند.

نتایج و تفسیر:

- **خروجی :**این سلول خروجی خاصی در کنسول نمایش نمی‌دهد، زیرا ربات تلگرامی را راه‌اندازی می‌کند و منتظر ورودی‌های کاربر می‌ماند. در صورت اجرای موفق، ربات در تلگرام فعال می‌شود و به دستورات start/، پیام‌های متنی، و صوتی پاسخ می‌دهد.
- **تحلیل عملکرد :**
  - **کاربرد عملی :**پیاده‌سازی ربات تلگرامی یک روش خلاقانه و کاربرپسند برای ارائه مدل به کاربران نهایی است. پشتیبانی از ورودی‌های متنی و صوتی، قابلیت استفاده را افزایش می‌دهد.
  - **بارگذاری منابع :**بارگذاری توکنایزر، تعبیه‌ها، و مدل به‌درستی انجام شده و امکان استفاده بلادرنگ از مدل را فراهم می‌کند.
  - **مدیریت ورودی‌ها :**مدیریت پیام‌های متنی ساده و مؤثر است. مدیریت پیام‌های صوتی با تبدیل صوت به متن و سپس طبقه‌بندی، ویژگی جذابی است، اما وابستگی به سرویس Google Speech Recognitionممکن است محدودیت‌هایی ایجاد کند.
  - **مدیریت خطاها :**تابع handle_voiceخطاهای تشخیص گفتار را به‌خوبی مدیریت می‌کند (مثل عدم درک صوت یا عدم دسترسی به سرویس). با این حال، تابع handle_textمی‌تواند برای ورودی‌های نامعتبر (مثل متن خالی) مقاوم‌تر شود.